

คู่มือการใช้งาน ev3link.py

คู่มือนี้อธิบายวิธีติดตั้งและใช้งาน `ev3link.py` ซึ่งเป็นไลบรารี Python สำหรับควบคุมบอร์ดขยาย STM32F103C8T6 ผ่าน EV3 sensor port (ev3dev)

1. สิ่งที่ต้องมีก่อนใช้งาน

- EV3 ที่ลง ev3dev เรียบร้อยแล้ว
- บอร์ด STM32 ที่ลง firmware เรียบร้อยแล้ว และต่อเข้ากับ EV3 sensor port ตามผังขาที่กำหนดไว้
- ไฟล์ `ev3link.py` วางอยู่ใน path เดียวกับสคริปต์ที่จะเรียกใช้
- ติดตั้ง pyserial (ปกติมีมาให้แล้วใน ev3dev แต่ถ้ายังไม่มีให้รัน):

```
pip3 install pyserial
```

2. การตั้งค่าเริ่มต้น (ทำครั้งเดียวต่อการเปิดเครื่อง/เสียบสาย)

ขั้นที่ 1 — สลับ sensor port เป็นโหมด raw UART

```
from ev3link import set_other_uart_mode
set_other_uart_mode("in1") # ถ้าเสียบพอร์ตอื่นให้เปลี่ยนเป็น "in2", "in3", "in4"
```

ฟังก์ชันนี้เขียนค่าไปที่ `/sys/class/lego-port/portN/mode` ให้อัตโนมัติ ต้องรันก่อนเปิดการเชื่อมต่อทุกครั้ง

ขั้นที่ 2 — หา path ของ tty ที่ถูกต้อง

ชื่อไฟล์ device จะไม่เหมือนกันในแต่ละเวอร์ชันของ ev3dev ให้ตรวจสอบเองก่อนใช้งานจริง:

```
ls -la /dev/tty*          # รันก่อนสลับโหมด (mode 1)
# ... สลับโหมดด้วย set_other_uart_mode() ...
ls -la /dev/tty*          # รันอีกครั้งหลังสลับโหมด (mode 2)
```

เทียบสอง output ดูว่ามี device ใหม่อะไรโผล่ขึ้นมา (พบบ่อยคือ `/dev/tty_in1` หรือ `/dev/ttyS1`) แล้วใช้ชื่อนั้นเป็นค่า `tty=` ตอนสร้าง `EV3Link`

3. เริ่มใช้งานแบบเร็ว (Quick Start)

```
from ev3link import EV3Link, set_other_uart_mode, DIR_FWD, DIR_STOP

set_other_uart_mode("in1")
```

```
with EV3Link(tty="/dev/tty_in1") as link:
    print("firmware version:", link.ping())

    link.set_motor(0, DIR_FWD, 200) # มอเตอร์ 0 เดินหน้า ความเร็ว 200/255
    link.set_servo(0, 90)           # เซอร์โว 0 ไปตำแหน่ง 90 องศา

    print("analog:", link.read_analog_all())
# ออกจาก with block แล้ว มอเตอร์จะหยุดอัตโนมัติ และปิด serial port ให้เอง
```

แนะนำให้ใช้ `with EV3Link(...) as link:` เสมอ เพราะจะสั่งหยุดมอเตอร์ทั้งหมดอัตโนมัติและปิด connection ให้เรียบร้อยแม้โปรแกรมจะ error หรือถูก Ctrl+C

4. คู่มืออ้างอิงฟังก์ชัน (API Reference)

สร้าง object

```
link = EV3Link(tty="/dev/tty_in1", baud=115200, timeout=0.3)
```

พารามิเตอร์ ความหมาย

<code>tty</code>	path ของ serial device (ดูวิธีหาใน หัวข้อ 2)
<code>baud</code>	ต้องตรงกับ firmware บนบอร์ด (ค่า default คือ 115200)
<code>timeout</code>	เวลาสูงสุด (วินาที) ที่รอคำตอบจากบอร์ดก่อนถือว่า timeout

`link.ping()`

เช็คบอร์ดตอบสนองหรือไม่ และดึงเลขเวอร์ชัน firmware

```
fw = link.ping() # เช่น 3
```

`link.set_motor(motor_id, direction, speed)`

สั่งมอเตอร์ขับเคลื่อน (dual-PWM, รองรับทั้งมอเตอร์ธรรมดาและมอเตอร์มี encoder)

พารามิเตอร์ ค่า

<code>motor_id</code>	0 หรือ 1
<code>direction</code>	DIR_STOP (0) / DIR_FWD (1) / DIR_REV (2)
<code>speed</code>	0-255 (ถูก clamp ให้อยู่ในช่วงนี้อัตโนมัติ)

```
link.set_motor(0, DIR_FWD, 200)
link.set_motor(1, DIR_REV, 100)
link.set_motor(0, DIR_STOP, 0)
```

หมายเหตุความปลอดภัย: ถ้าไม่ส่งคำสั่งใดๆ ไปที่บอร์ดนานเกิน 1 วินาที firmware จะหยุดมอเตอร์ทั้งหมด
ฮัดโนมิตี (watchdog) ดังนั้นถ้าต้องการให้มอเตอร์วิ่งต่อเนื่องนานๆ ต้องส่งคำสั่งซ้ำเป็นระยะ (เช่นทุก 200-500ms)
 แม้ค่าจะเหมือนเดิมก็ตาม

`link.set_servo(servo_id, angle)`

สั่งเซอร์โวไปยังตำแหน่งองศาที่กำหนด

พารามิเตอร์ ค่า

`servo_id` 0-5 (มีเซอร์โวทั้งหมด 6 ตัว)

`angle` 0-180 องศา (ถูก clamp ฮัดโนมิตี)

```
link.set_servo(0, 0)
link.set_servo(0, 180)
link.set_servo(0, 90)
```

`link.read_analog(sensor_id)`

อ่านค่าจากขา analog หนึ่งช่อง

พารามิเตอร์ ค่า

`sensor_id` 0-3

คืนค่า: จำนวนเต็ม 0-4095 (ความละเอียด ADC 12-bit) — เป็นค่าดิบ ไม่ได้แปลงหน่วย ต้องคำนวณแปลงเป็นแรงดัน/ค่า
 จริงเองตามชนิดเซนเซอร์ที่ต่ออยู่

```
raw = link.read_analog(0)
voltage = raw / 4095 * 3.3    # ตัวอย่างแปลงเป็นโวลต์ (อ้างอิง 3.3V)
```

`link.read_analog_all()`

อ่านค่า analog ทั้ง 4 ช่องพร้อมกันในคำสั่งเดียว (เร็วกว่าเรียก `read_analog()` 4 ครั้ง)

คืนค่า: tuple 4 ค่า (`ch0`, `ch1`, `ch2`, `ch3`)

```
v0, v1, v2, v3 = link.read_analog_all()
```

`link.i2c_write(channel, addr, data)`

เขียนข้อมูลไปยังอุปกรณ์ I2C ที่ต่อกับบอร์ด

พารามิเตอร์ **ค่า**

<code>channel</code>	0 (บัส I2C1) หรือ 1 (บัส I2C2)
<code>addr</code>	7-bit I2C address ของอุปกรณ์ปลายทาง (เช่น 0x1D)
<code>data</code>	<code>bytes</code> หรือ <code>list</code> ของ byte ที่จะเขียน (สูงสุดรวมกัน 29 byte ต่อคำสั่ง)

```
link.i2c_write(channel=0, addr=0x1D, data=[0x20, 0x0F])    # เขียน 2 byte ไปที่
register/คำสั่งของอุปกรณ์
```

`link.i2c_read(channel, addr, length)`

อ่านข้อมูลจากอุปกรณ์ I2C

พารามิเตอร์ **ค่า**

<code>channel</code>	0 หรือ 1
<code>addr</code>	7-bit I2C address
<code>length</code>	จำนวน byte ที่ต้องการอ่าน (สูงสุด 32)

คืนค่า: `bytes` ยาว `length`

```
data = link.i2c_read(channel=0, addr=0x1D, length=6)
print(list(data))
```

ก่อนอ่าน มักต้อง `i2c_write()` เพื่อบอกอุปกรณ์ว่าต้องการอ่าน register ไหนก่อนเสมอ (ขึ้นกับ datasheet ของอุปกรณ์แต่ละตัว)

`link.read_encoder(motor_id) / link.reset_encoder(motor_id)`

อ่าน/รีเซ็ตค่านับ encoder ของมอเตอร์ (ใช้ได้เฉพาะมอเตอร์ที่ต่อ encoder จริงเท่านั้น มอเตอร์ที่ไม่มี encoder จะได้ค่า 0 เสมอ)

พารามิเตอร์ ค่า

`motor_id` 0 หรือ 1

คืนค่า (`read_encoder`): จำนวนเต็มมีเครื่องหมาย (นับสะสม ไม่จำกัดรอบ หมุนกลับทิศได้ค่าลบ)

```
link.reset_encoder(0)
link.set_motor(0, DIR_FWD, 150)
time.sleep(0.5)
link.set_motor(0, DIR_STOP, 0)
print("หมุนไปทั้งหมด:", link.read_encoder(0), "tick")
```

`link.stop_all()`

สั่งหยุดมอเตอร์ทั้งหมดทันที (servo ไม่ได้รับผลกระทบ)

```
link.stop_all()
```

`link.get_status()`

ตรวจสอบสถานะบอร์ด

คืนค่า: tuple (`ok`, `uptime_ms`) — `ok` เป็น 1 ถ้าปกติ, `uptime_ms` คือเวลาที่บอร์ดทำงานมาแล้ว (มิลลิวินาที) นับตั้งแต่ล่าสุดที่ reset

```
ok, uptime = link.get_status()
print("uptime:", uptime, "ms")
```

`link.close()`

สั่งหยุดมอเตอร์ทั้งหมดแล้วปิด serial port — **เรียกเองไม่จำเป็นถ้าใช้ `with EV3Link(...)` as `link`:**

```
link.close()
```

5. ค่าคงที่ (Constants) ที่ import ใช้ได้

```
from ev3link import DIR_STOP, DIR_FWD, DIR_REV
```

ชื่อ	ค่า	ความหมาย
DIR_STOP	0	หยุดมอเตอร์
DIR_FWD	1	เดินหน้า
DIR_REV	2	ถอยหลัง

6. การจัดการ Error

ทุกฟังก์ชันที่คุยกับบอร์ด อาจโยน exception `EV3LinkError` ถ้า:

- บอร์ดไม่ตอบสนอง (สายหลุด/ไฟไม่เข้า/ baud ไม่ตรง) → ข้อความจะบอกว่า "timeout ..."
- บอร์ดตอบกลับมว่าคำสั่งผิดพลาด (NACK) → ข้อความจะบอกชื่อ error พร้อมรหัส

รหัส error จากบอร์ด	ความหมาย	สาเหตุที่พบบ่อย
BAD_CHECKSUM	checksum ไม่ตรง	สายรบกวน/สัญญาณไม่นิ่ง
BAD_LEN	ความยาว payload ผิด	เรียกฟังก์ชันผิดพารามิเตอร์ (ปกติไม่เกิดถ้าใช้ library ตามคู่มือนี้)
UNKNOWN_CMD	คำสั่งที่ไม่รู้จัก	firmware กับ library คนละเวอร์ชันกัน
I2C_FAIL	เขียน/อ่าน I2C ไม่สำเร็จ	ต่อสายผิด, address ผิด, อุปกรณ์ไม่ตอบสนอง
BAD_PARAM	พารามิเตอร์เกินขอบเขต	เช่น <code>motor_id=2, servo_id=8, channel=5</code>

ตัวอย่างการดักจับ error:

```
from ev3link import EV3Link, EV3LinkError, set_other_uart_mode

set_other_uart_mode("in1")

try:
    with EV3Link(tty="/dev/tty_in1") as link:
        link.ping()
        data = link.i2c_read(channel=0, addr=0x1D, length=6)
        print(data)
except EV3LinkError as e:
    print("เกิดข้อผิดพลาดในการสื่อสารกับบอร์ด:", e)
```

การเรียกฟังก์ชันแต่ละครั้งจะพยายามส่งซ้ำอัตโนมัติสูงสุด 2 ครั้งก่อนโยน exception ให้ (กันปัญหาสัญญาณรบกวนเป็นครั้งคราว) ถ้ายัง error อยู่หลังจากลองซ้ำแล้ว แปลว่าน่าจะเป็นปัญหาจริง (สาย/ไฟ/ตั้งค่า) ควรตรวจสอบฮาร์ดแวร์

7. แก้ปัญหาเบื้องต้น (Troubleshooting)

อาการ	สิ่งที่ควรตรวจสอบ
<code>serial.serialutil.SerialException</code> ดอนสร้าง <code>EV3Link()</code>	path <code>tty=</code> ผิด — ทำตามหัวข้อ 2 ใหม่เพื่อหา path ที่ถูกต้อง
<code>EV3LinkError: timeout waiting for STX</code> ทุกครั้ง	บอร์ดไม่ตอบเลย เช็ค: ไฟเข้าบอร์ดหรือยัง, ต่อสาย TX/RX สลับกันหรือไม่ (TX ของ EV3 ต้องเข้า RX ของบอร์ด และกลับกัน), GND ต่อรวมกันหรือยัง
ได้ค่า timeout เป็นบางครั้ง	สายหลวม/ยาวเกินไป/มีสัญญาณรบกวน ลองต่อสายให้สั้นลงหรือหุ้ม shield
<code>EV3LinkError: STM32 NACK: BAD_CHECKSUM</code>	คล้ายข้อข้างบน (สัญญาณรบกวน) หรือ baud rate ไม่ตรงกันระหว่าง firmware กับ <code>baud=</code> ที่ตั้งไว้
<code>EV3LinkError: STM32 NACK: I2C_FAIL</code>	เช็ค address I2C ให้ตรง (บางอุปกรณ์ระบุ address แบบ 8-bit ในเอกสาร ต้อง shift ขวา 1 บิตก่อนใช้กับ library นี้), เช็คสาย SDA/SCL และ pull-up resistor
มอเตอร์/เซอร์โวไม่ขยับเลยแต่ <code>ping()</code> ผ่าน	เช็คการต่อสายไฟเลี้ยงมอเตอร์/เซอร์โวแยกต่างหาก (บอร์ดส่งได้แต่ไม่ได้จ่ายไฟกำลังให้มอเตอร์)
มอเตอร์วิ่งแป๊บเดียวแล้วหยุดเอง	โดน watchdog ตัดเพราะไม่ได้ส่งคำสั่งซ้ำภายใน 1 วินาที ให้ส่ง <code>set_motor()</code> ซ้ำเป็นระยะระหว่างที่ต้องการให้วิ่งต่อเนื่อง

8. ตัวอย่างโปรแกรมแบบเต็ม

ดูไฟล์ `example_usage.py` ที่มากับคู่มือนี้ สำหรับตัวอย่างที่ใช้งานครบทุกฟังก์ชัน (มอเตอร์, เซอร์โว, analog, I2C, encoder) พร้อมกัน